

Knowledge Graph Construction and Analysis

Knowledge Graph in Full Detail

Correct heading:

Knowledge Graph

A **Knowledge Graph** is a way of representing information as a **network of connected things**.

In simple words:

A knowledge graph stores **entities** and the **relationships between them**.

Example:

Virat Kohli → **plays for** → **India**

Virat Kohli → **born in** → **Delhi**

India → **has capital** → **New Delhi**

Here:

- **Virat Kohli, India, Delhi, New Delhi** are **entities**
- **plays for, born in, has capital** are **relationships**

So instead of storing information only in rows and columns, a knowledge graph stores information like a **connected map of knowledge**.

1. What is a Knowledge Graph?

A **Knowledge Graph** is a graph-based data structure that connects real-world objects, concepts, or entities using meaningful relationships.

It has mainly two parts:

Part	Meaning	Example
Node / Entity	Any object, person, place, concept, event	Person, Company, Disease, Movie

Edge / Relationship	Connection between two entities	works_at, located_in, causes
Property / Attribute	Extra information about entity or relation	age, date, weight, role

Example:

Antriksh → studies → Machine Learning
Machine Learning → is part of → Artificial Intelligence
Artificial Intelligence → used in → Chatbots

This forms a graph.

2. Why is it called a “Graph”?

Because it follows the graph structure from mathematics/computer science.

A graph has:

Nodes + Edges

Example:

[Person] ---- works_at ----> [Company]
[Company] ---- located_in ----> [City]
[City] ---- part_of ----> [Country]

So the system can understand connections like:

This person works at a company, that company is in a city, and that city is in a country.

This makes knowledge graphs very powerful for reasoning.

3. Basic Example

Suppose we have this sentence:

“Elon Musk founded SpaceX in 2002.”

From this sentence, a knowledge graph can extract:

Elon Musk $\rightarrow$ founded $\rightarrow$ SpaceX

SpaceX $\rightarrow$ founded_in $\rightarrow$ 2002

Elon Musk $\rightarrow$ role $\rightarrow$ Founder

Graph form:

[Elon Musk] ---- founded ----> [SpaceX]

[SpaceX] ---- founded_in ----> [2002]

This is called a **triple**.

4. What is a Triple?

A triple is the basic unit of a knowledge graph.

It has 3 parts:

Subject $\rightarrow$ Predicate $\rightarrow$ Object

Example:

Elon Musk $\rightarrow$ founded $\rightarrow$ SpaceX

Here:

Component	Value
Subject	Elon Musk

Predicate founded

Object SpaceX

More examples:

Apple → produces → iPhone

iPhone → belongs_to → Smartphone

Tesla → CEO → Elon Musk

Delhi → capital_of → India

Knowledge graphs are mostly built using millions or billions of such triples.

5. Main Components of a Knowledge Graph

1. Entities

Entities are the main objects.

Examples:

Person

Movie

Company

Disease

Medicine

Location

Product

Research Paper

Author

In your project, entities could be:

Machine Learning

Knowledge Graph

Natural Language Processing

BERT

Neural Network

Dataset
Research Paper

2. Relationships

Relationships define how two entities are connected.

Examples:

Person → works_at → Company
Disease → treated_by → Medicine
Movie → directed_by → Director
Paper → written_by → Author
Model → trained_on → Dataset

Relationships are the most important part because they give meaning to the graph.

3. Properties

Properties are additional details.

Example:

Person: Virat Kohli
age: 35
country: India
profession: Cricketer

For a knowledge graph:

[Virat Kohli]
- age: 35
- nationality: Indian
- profession: Cricketer

Relations can also have properties:

Virat Kohli → plays_for → India
since: 2008
role: batsman

4. Schema / Ontology

This is the structure or rulebook of the knowledge graph.

It defines:

What types of entities are allowed?
What relationships are allowed?
Which entity can connect to which entity?

Example:

Person can work_at Company
Company can be located_in City
Disease can be treated_by Medicine

Without schema, the graph becomes messy.

Example of bad graph:

Python → eats → Delhi

This relationship makes no sense.

A schema prevents such wrong connections.

6. Knowledge Graph vs Normal Database

A normal database stores data in tables.

Example table:

Person	Company	City
Elon Musk	SpaceX	Hawthorne
Sundar Pichai	Google	Mountain View

But a knowledge graph stores the same data as connected relationships:

Elon Musk → founded → SpaceX
 SpaceX → located_in → Hawthorne

Sundar Pichai → CEO_of → Google
 Google → located_in → Mountain View

Difference

Normal Database	Knowledge Graph
Stores rows and columns	Stores entities and relationships
Good for fixed structured data	Good for connected knowledge
Hard to find indirect relationships	Easy to find indirect relationships
Less semantic meaning	More semantic meaning
Used in banking, ERP, records	Used in search, recommendation, AI, RAG

7. Why Knowledge Graph is Useful

Knowledge graphs are useful because they help machines understand relationships.

For example, if the graph has:

A → friend_of → B
 B → works_at → Google
 Google → located_in → USA

Then the system can infer:

A knows someone who works in USA.

This is why knowledge graphs are used in:

- Google Search
 - Recommendation systems
 - Chatbots
 - Question answering
 - Fraud detection
 - Medical diagnosis
 - Research paper analysis
 - RAG systems
 - Entity relationship extraction
 - Semantic search
-

8. Real-Life Examples of Knowledge Graphs

Google Knowledge Graph

When you search:

Albert Einstein

Google shows:

Born: Germany

Known for: Theory of Relativity

Spouse: Elsa Einstein

Awards: Nobel Prize

This comes from a knowledge graph.

Google connects:

Albert Einstein → born_in → Germany

Albert Einstein → developed → Theory of Relativity

Albert Einstein → won → Nobel Prize

Netflix / Amazon Recommendation

Example:

User → watched → Inception

Inception → genre → Sci-Fi

Inception → directed_by → Christopher Nolan

Christopher Nolan → directed → Interstellar

Then Netflix may recommend **Interstellar**.

Because the graph knows both movies are connected by genre and director.

Medical Knowledge Graph

Example:

Fever → symptom_of → Dengue

Dengue → caused_by → Virus

Dengue → treated_by → Supportive Care

Paracetamol → reduces → Fever

This helps in medical question answering.

9. How Knowledge Graph is Built

A knowledge graph can be built manually or automatically.

Step 1: Collect Data

Data can come from:

Text documents

PDFs
Web pages
CSV files
Research papers
Wikipedia
Databases
APIs

Example text:

“BERT is a transformer-based model developed by Google.”

Step 2: Extract Entities

From the sentence:

“BERT is a transformer-based model developed by Google.”

Entities are:

BERT
Transformer
Google

This is called **Named Entity Recognition**, or **NER**.

Step 3: Extract Relationships

Relationships are:

BERT → is_based_on → Transformer
BERT → developed_by → Google

This is called **Relation Extraction**.

Step 4: Create Triples

Triples:

(BERT, is_based_on, Transformer)

(BERT, developed_by, Google)

Step 5: Store the Graph

You can store it using:

NetworkX

Neo4j

RDF triples

GraphDB

TigerGraph

ArangoDB

JSON

For a small project, you can use:

NetworkX + Python

For a bigger production project, you can use:

Neo4j

Step 6: Query the Graph

You can ask questions like:

Who developed BERT?

Which models are based on Transformer?

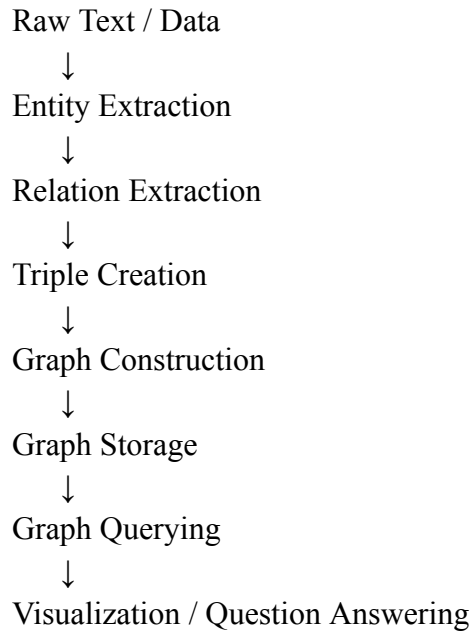
Which datasets are used by this model?

Example answer:

BERT was developed by Google.

10. Knowledge Graph Pipeline

A typical knowledge graph project looks like this:



For example:

Input:

"BERT was developed by Google and is based on Transformer architecture."

Output:

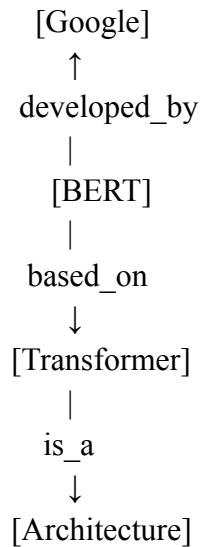
BERT → developed_by → Google

BERT → based_on → Transformer

Transformer → is_a → Architecture

11. Knowledge Graph Visualization

A knowledge graph can be visualized like this:



This visual structure helps us understand data better.

12. Knowledge Graph in AI and Machine Learning

Knowledge graphs are very useful in AI because they give **structured knowledge** to models.

Normal ML models learn from data, but they often do not understand real-world relationships clearly.

Knowledge graphs help by giving explicit relationships.

Example:

Cancer → treated_by → Chemotherapy

Chemotherapy → has_side_effect → Hair Loss

An AI system can use this to answer:

What are possible side effects of cancer treatment?

Answer:

Hair loss may be a side effect of chemotherapy.

13. Knowledge Graph in RAG

RAG means:

Retrieval-Augmented Generation

Normal RAG retrieves text chunks from documents.

Knowledge Graph RAG retrieves connected facts.

Example user asks:

“Which author worked on BERT and later contributed to Transformer models?”

A vector database may only retrieve similar text.

But a knowledge graph can follow connections:

Author → wrote → Paper

Paper → about → BERT

Author → contributed_to → Transformer

So KG-RAG gives more accurate and explainable answers.

14. Knowledge Graph vs Semantic Graph

This is important.

Semantic Graph

A semantic graph connects concepts based on meaning or similarity.

Example:

Dog — similar_to — Cat

Machine Learning — related_to — Deep Learning

BERT — similar_to — RoBERTa

It may use embeddings or semantic similarity.

Knowledge Graph

A knowledge graph connects entities using factual relationships.

Example:

BERT → developed_by → Google

RoBERTa → developed_by → Facebook AI

BERT → uses → Transformer

Simple Difference

Semantic Graph	Knowledge Graph
Based on meaning/similarity	Based on facts/relations
Edges may mean “similar to”	Edges have exact meaning
Often built using embeddings	Built using triples
Good for semantic search	Good for reasoning and facts

Example:

BERT — similar_to — RoBERTa

This is semantic graph.

BERT → developed_by → Google

This is knowledge graph.

15. Knowledge Graph vs Ontology

Ontology is the rulebook.

Knowledge graph is the actual data.

Example ontology:

Person can work_at Company
Company can be located_in City

Example knowledge graph:

Sundar Pichai → works_at → Google
Google → located_in → California

So:

Ontology = structure
Knowledge Graph = actual connected data

16. Knowledge Graph Embeddings

Knowledge graph embeddings convert entities and relationships into vectors.

Why?

Because machine learning models cannot directly understand graph nodes and edges.

So we convert them into numbers.

Example:

BERT = [0.12, 0.45, 0.78, ...]
Google = [0.22, 0.61, 0.19, ...]
developed_by = [0.31, 0.17, 0.89, ...]

Popular KG embedding methods:

TransE
TransH
TransR
DistMult
ComplEx
RotatE
Graph Neural Networks

These are used for:

Link prediction
Entity similarity
Recommendation
Missing relation prediction

Example:

Known facts:

BERT → developed_by → Google
RoBERTa → developed_by → Facebook
GPT → developed_by → ?

The model may predict:

GPT → developed_by → OpenAI

17. What is Link Prediction?

Link prediction means predicting missing relationships.

Example graph has:

BERT → based_on → Transformer
GPT → based_on → Transformer
BERT → developed_by → Google
GPT → developed_by → ?

The model tries to predict the missing edge:

GPT → developed_by → OpenAI

This is useful when the knowledge graph is incomplete.

18. Tools Used for Knowledge Graphs

For small projects

Python
NetworkX
Pandas
Matplotlib
spaCy
NLTK
Hugging Face Transformers

For graph database

Neo4j
GraphDB
ArangoDB
TigerGraph
Amazon Neptune

For NLP extraction

spaCy
BERT
RoBERTa
XLM-R
OpenAI API
Gemini API
LangChain
LlamaIndex

For visualization

NetworkX
PyVis
Neo4j Bloom
Gephi

19. Simple Python Example

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()

G.add_edge("BERT", "Google", relation="developed_by")
G.add_edge("BERT", "Transformer", relation="based_on")
G.add_edge("Transformer", "Attention", relation="uses")
G.add_edge("RoBERTa", "BERT", relation="improves")

pos = nx.spring_layout(G)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_size=3000,
    node_color="lightblue",
    font_size=10,
    arrows=True
)

edge_labels = nx.get_edge_attributes(G, "relation")
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)

plt.show()
```

This creates a simple knowledge graph.

20. Example Knowledge Graph Project

Project title:

Knowledge Graph Construction and Analysis

Objective:

To extract entities and relationships from text data and represent them as a graph for better knowledge representation, visualization, and querying.

Pipeline:

Input Text
↓
Preprocessing
↓
Entity Extraction
↓
Relationship Extraction
↓
Triple Creation
↓
Knowledge Graph Construction
↓
Visualization
↓
Analysis

Example input:

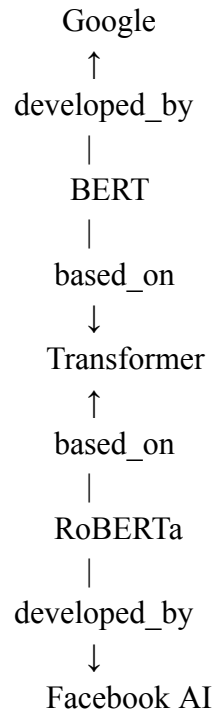
BERT is developed by Google.
RoBERTa is developed by Facebook AI.
Both are based on Transformer architecture.

Triples:

BERT → developed_by → Google
RoBERTa → developed_by → Facebook AI
BERT → based_on → Transformer
RoBERTa → based_on → Transformer

Transformer → type → Architecture

Graph:



21. Advantages of Knowledge Graphs

Knowledge graphs are useful because they:

- Represent information clearly
 - Show relationships between entities
 - Help in reasoning
 - Improve search results
 - Support question answering
 - Make AI systems more explainable
 - Can combine structured and unstructured data
 - Help in recommendation systems
-

22. Limitations of Knowledge Graphs

They also have some problems:

- Hard to build manually
- Automatic extraction can make mistakes
- Entity names can be confusing
- Same entity may appear in different forms
- Large graphs need powerful storage
- Relation extraction is difficult
- Keeping the graph updated is challenging

Example problem:

- USA
- United States
- United States of America
- America

All may refer to the same entity.

This is called **entity resolution**.

23. Entity Resolution

Entity resolution means identifying that different names refer to the same thing.

Example:

- IBM
- International Business Machines
- IBM Corp.

All refer to the same company.

In a knowledge graph, we should merge them into one entity.

Otherwise, the graph becomes duplicate and messy.

24. Important Terms

Term	Meaning
Entity	Object or concept
Node	Graph representation of entity
Edge	Relationship between entities
Triple	Subject-predicate-object format
Ontology	Rulebook/schema
Relation Extraction	Finding relationships from text
Entity Resolution	Merging same entities
Link Prediction	Predicting missing relationships
RDF	Standard triple format
SPARQL	Query language for RDF graphs
Cypher	Query language for Neo4j

25. Final Simple Explanation

A **Knowledge Graph** is a connected structure of facts.

It represents knowledge like this:

Entity → Relationship → Entity

Example:

BERT → developed_by → Google

BERT → uses → Transformer

Transformer → uses → Attention

It helps machines understand not only **what things are**, but also **how they are connected**.

In simple words:

A knowledge graph is like a smart mind map where every connection has a clear meaning.

Best project explanation:

In this project, we construct a knowledge graph by extracting entities and relationships from text data. The extracted facts are represented as triples and visualized as a graph. This helps in understanding hidden connections between concepts, improving information retrieval, and enabling question answering over structured knowledge.